

The Inference Serving Problem

Deep Dive into SGLang — Chapter 2 (Foundations)

Yin Yang

Foundation Books Project

Where this chapter sits

This is the chapter that turns “run a model” into “run a service.”

- Volume I is organized around a stateful service loop: scheduling, memory, prefix reuse, measurement.
- Before any SGLang mechanism appears, we state the underlying **control problem** the runtime is solving.
- Everything later — continuous batching, the KV cache, RadixAttention — is an answer to a question we pose here.

Goal: by the end you can look at a serving decision and say what the runtime can see, what it may run, and which request pays if it waits.

The control problem

The variables that recur

Why the model is honest

The control problem

Serving is a stateful workload, not one forward pass

- Many users send request streams concurrently.
- New requests arrive *while* older ones are still decoding.
- Users expect partial tokens to *stream* back in order.
- Expensive accelerators must stay busy — without letting token state exhaust fast memory (GPU HBM).

At each step the runtime faces a continuously updated dispatch problem:

what work is waiting? what resources exist? what constraints bind? what is the objective?

The running trace: four requests, three pressures

Request	Arrival	Prompt	Output	Pressure
A / r_{pan}	0 ms	64	8	short chat turn
B	5 ms	4096	2	prompt-heavy
C	8 ms	256	120	decode-heavy
D	12 ms	1024	16	shares a long prefix with B

The arrival order hides the real story. B is expensive up front, C is expensive for a long time, and D is cheap — but only if the runtime recognizes its prefix is already resident.

The variables that recur

Five quantities the rest of the book keeps using

Request state the per-request record that survives across steps.

Prefill vs. decode two kinds of model work with different shapes.

Token state resident per-token model state that future positions read — and what fills up fast memory.

Effective prompt work prompt cost *after* prefix reuse, not raw prompt length.

Latency & throughput TTFT, ITL, end-to-end, and throughput — the user-visible objective.

A scheduler that counts requests instead of these quantities makes the wrong choice on B, C, and D.

The online serving decision

At time t the policy π maps *visible* state to a *feasible* action:

$$u_t = \pi(\mathcal{X}_t) \in \mathcal{U}_{\text{feas}}(\mathcal{X}_t).$$

- $\mathcal{X}_t$: pending requests, resident KV pages, budgets, live positions — **no future arrivals**.
- $\mathcal{U}_{\text{feas}}$: actions that fit slots, token budgets, fast memory, and compatibility.
- Feasibility draws the legal boundary; the policy *ranks* legal actions by the objective.

Actions: **admit** prompt work, **advance** a decode token, **delay**, or **release**.

From concept to code: “run prefill first, else decode”

The whole online decision has a concrete home in the scheduler loop.

```
from python/sglang/srt/managers/scheduler.py

new_batch = self.get_new_batch_prefill()
...
if new_batch is not None:
    # Run prefill first if possible
    ret = new_batch
else:
    # Run decode (skip for prefill-only batches)
    if (not self.running_batch.is_empty()
        and not self.running_batch.is_prefill_only):
        self.running_batch = self.update_running_batch(self.running_batch)
        ret = self.running_batch if not ... else None
```

One step = try to assemble a feasible prefill batch; otherwise advance decode. The chapter's definitions explain every word of that comment.

Why the model is honest

The model is tested, not just asserted

- **Empirical anchors:** benchmark scenarios on real hardware (Jetson Thor, Qwen2.5-0.5B) move the measured surface in the predicted direction — e.g. prefix reuse on/off, prompt-heavy vs. decode-heavy.
- **Source pressure:** the variables we introduce are exactly the ones SGLang's scheduler was forced to track as it evolved.

We only make claims as large as the metric, workload, and source support.

What to carry forward

1. Serving is an **online control problem** under finite accelerator and memory budgets.
2. Reason about **token work, resident state, prefix reuse, and latency** — never request count alone.
3. **Prefill, decode, and token state** stay separate quantities throughout.
4. Unfinished requests' **future token state stays accounted for**, even when this step is one slice.
5. Every concept has a **home in the SGLang source**; that link is the spine of this book.

Next: Chapter 3 turns this decision into a ledger of scheduled rows, attention-pair units, and KV slots.